sltoo – Traceability

Kristoffer Nordström

May 31, 2021

Introduction

Storytime

- The Foundling
- Word is also a Hammer
- The Spanish Review
- The Blame Game

The Foundling

Once upon a time a we inherited from another contractor a project that has been dormant for a couple of years.

It came with one document that contained all requirements and use-cases. It appeared to have been exported from some web-based tool.

Question: How do we ensure that all requirements and documentation is shipped with the code for future use?

Word is also a Hammer

A different project had one test report with a traceability matrix. It's perfectly fine to create one manually. If the requirements change it'll be hard to figure out the minimal delta-test.

Obviously the requirements change more than once. Hence we want this fully automated.

The Spanish Review

I've once had the pleasure to review multiple specifications for an ESA contractor. Unfortunately, they didn't provide a consistent set of documents.

We have CI/CD for software, why don't we include our documents?

The Blame Game

We can branch software files and have tools to review merges/changes.

Software is just text, requirement specifications too. Can we use the same tools, e.g., running `git blame` on the spec?

Theory

Non-locality of comments

Perhaps the biggest problem with code comments are their non-locality

Dragons

```
# Here be dragons, don't forget to clean up later
some_func()
...
cleanup()
```

Changes to `some_func()` will not propagate back to the above snippet, let alone the clean-up.

Non-locality of comments (II)

```java
// Always returns true.
public boolean isAvailable() {
    return false;
}
```

Traceability Directions

- Traceability from specification items
  - Forward from requirements specification to dependant documents
  - Backwards from verification to specifications

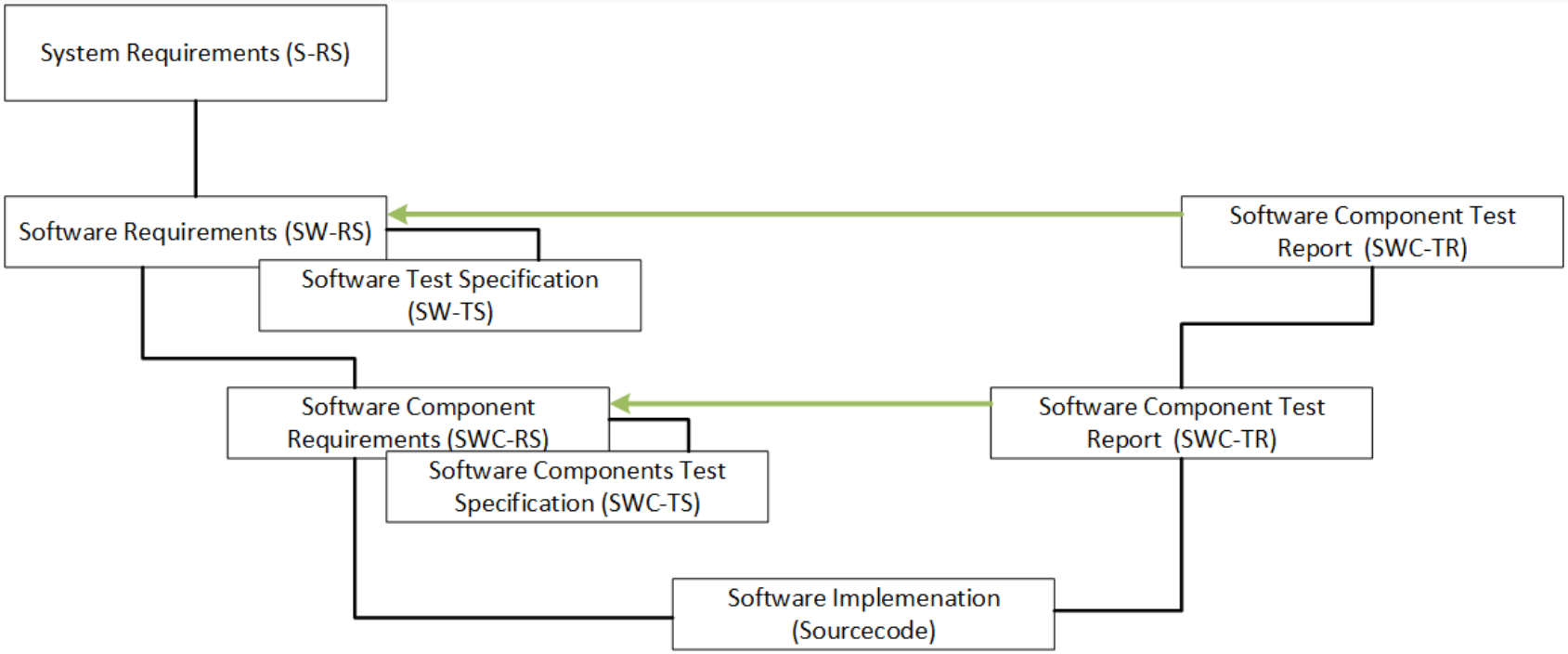

Figure 1: Traceability Overview

Traceability

- *Requirement A* says do `A`
- *Implementation a* says implemented `A`
  - What if `A` changes?
  - *A* knows nothing of *a*
  - Can be tedious manually

Proposed Solution

- *Requirement A-1.0* says do `A`
- *Implementation a* says implemented `A-1.0`
  - *A-1.0* changes to *A-2.0* do $\hat{\mathtt{A}}$
  - Use hashes instead of semantic versioning
  - Calculated automatically

Practice

Baseline

- Based on rmtoo
  - File-based requirements tracking tool
  - Requirements stored in text-files
  - group requirements by topics
  - document and requirements LaTeXbased
    - Customizable with templates
- Manage with your favourite VCS
  - Merge changes from various sources
- Excel import/export
- Traceability

Example Requirement

```
Name: VCD Writer Inputs
Topic: ReqsDocument
Description: The output from ...
Rationale: Make the process as ...
Status: external
Owner: development
Effort estimation: 1
Invented on: 2020-05-30
Invented by: default
Type: requirement
```

VCD Writer Inputs SW-AS-501
The output from *AggregatorInterface* can be fed into the *VCD Writer* to create an identical output file.
Rationale: Make the process as transparent as possible.
Depends on: SW-AS-500

Id:	SW-AS-501	Priority:	0.0	Owner:	development
Invented on:	2020-05-30	Invented by:	default	Status:	passed
Class:	detailable	Hash:	f8d68d11		

Figure 2: Example Requirement

Hash is calculated over *Name*, *Description* and *Verification Method*

Testing the Example Requirement

- Requirement ID: `SW-AS-501`
- Hash: `F8D68D11`
- Status: *external*

Test Code

```python
def test_read_write_engines(record_property, dummy_vcd_file):
    """Write-back from read file, equal output"""
    record_property('req', 'SW-AS-501-f8d68d11')
    record_property('req', 'SW-AS-500-4c1a395a')
    ...
    assert filecmp.cmp(dummy_vcd_file, ofile)
```

xUnit Output

```xml
<testcase classname="tests.test_io_manager" file="tests/test_io_manager.py" line="20" name="test_read_writ
  <properties>
    <property name="req" value="SW-AS-501-f8d68d11"/>
    <property name="req" value="SW-AS-500-4c1a395a"/>
  </properties>
</testcase>
```

Traceability Matrix

Traceability Matrix

Table A.1: Traceability Matrix Table

Req. ID	Status	UT
SW-AS-1	passed	passed
SW-AS-100	open	open
SW-AS-101	open	open
SW-AS-102	open	open
SW-AS-200	passed	passed
SW-AS-201	passed	passed
SW-AS-202	passed	passed
SW-AS-300	passed	passed
SW-AS-301	open	open
SW-AS-302	open	open
SW-AS-500	passed	passed
SW-AS-501	passed	passed

CI/CD

- Integration for every output document
- Match *open* and/or *failed* issues
- Example for *failed* issues

```
bash -ec 'test "$(grep -c failed \
   arch/artifacts/tracematrix.tex)" -eq "0"'
```

Versioning

- Every document has a release process

```
git tag -a RS/1A
git describe $(git log -n 1 --format=%H -- docs/reqs)
```

Output good and tainted:

```
$ RS/1A-8-g76b3ffe - 76b3ffe4
$ RS/1A - 0aec3ad0
```

Conclusion

Storytime Revisited

- Requirements shipped with code ✓
- Traceability matrix automated ✓
- Continuously updated documentation ✓
  - Document Versioning ✓
- `git blame` and `gitk`? ✓

Storytime Revisited

- Requirements shipped with code ✓
- Traceability matrix automated ✓
- Continuously updated documentation ✓
  - Document Versioning ✓
- `git blame` and `gitk`? ✓

Your Jira workspace will be gone

Questions?

Appendix

Excel Support Rationale

- Good-enough GUI
- Understood by everyone
- The *Truth* is still in your repository
  - Import from Excel to repository
  - Export every build to a new Excel file
  - Templating for branding

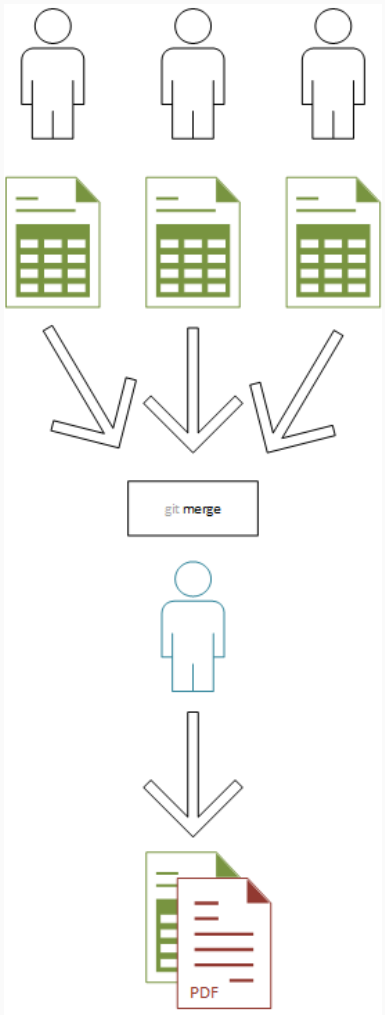

Figure 3: Workflow

Introduction Presentations

An introduction presentation into *rmtoo* and with more details.

Traceability Rationale

- Traceability for the given requirements
- Bring code and documentation into same repository
- Integrate into build-system
  - Detect upstream changes to requirements
  - Quickly identify affected code-regions
- No silver bullet for verification

Results

The status *external* will yield the following results:

- *open*
  - No matching requirement ID
- *passed*
  - Matching requirement ID
  - All hashes match
  - Unit-tests passed
- *failed*
  - Matching requirement ID
  - Some/all hashes didn't match, or
  - Unit-tests haven't passed

Installation

```
pip3 install sltoo==24.5b5
```

Traceability features are in the beta releases.

Final Thoughts

- Never test against your requirements
- Always write some form of test specification

Specification Hash

- SHA256 hash calculated over sum of
  - Description,
  - Titel, and
  - Verification Method (if available)
- Rationale is only informative